AuthentiMark — Implementation & Ethics Guide

This document explains the technical implementation, technology choices, and ethical considerations of the **AuthentiMark** system. It is written in simple, easy-to-understand language to serve as a guide for both technical developers and non-technical stakeholders.

---

1. Tech Stack & Purpose

The AuthentiMark system is divided into two parts: the **Backend** (the brain that runs the AI calculations) and the **Frontend** (the visual interface you interact with in the browser).

The Backend (Python)

- **FastAPI**: A modern, high-performance web framework. Its purpose is to handle user requests (like "generate an image" or "check this image") and send back the results quickly.
- **PyTorch**: A deep learning library. We use it to load our trained models (`ae_latest.pt`, `vae_latest.pt`, and `detector_latest.pt`) and run the math to embed or detect watermarks.
- **Pillow (PIL)**: A library for processing images. We use it to load, resize, compress, crop, and blur images.
- **Scikit-Image & Numpy**: Numerical tools used to calculate image similarity metrics (PSNR and SSIM) and inject random noise during attack simulations.
- **Uvicorn**: A fast server that hosts the FastAPI application locally.

The Frontend (React, Vite, & Tailwind)

- **React**: A component-based JavaScript library. We use it to manage user inputs, button clicks, and files uploaded by the user.
- **Vite**: A fast build tool that bundles and runs the React code in the browser.
- **Tailwind CSS**: A styling framework. We use it to design the Light and Dark mode interfaces, giving the application its clean, rounded-corner appearance.

---

2. Implementation Overview

How the Backend Works

1. **Startup Check**: When the backend server starts, it loads all three trained models (`AE`, `VAE`, and `Detector`) into the computer's memory (RAM) once. This ensures that when a user requests a watermark check, the server responds instantly without loading weights from the disk each time.

2. **Embedding watermarks**: When you upload an image to watermark, the backend downsamples it to 128 \times 128 pixels. It generates a random 32-bit key (a secret sequence of 0s and 1s) and injects it into the image pixels using the Autoencoder.
3. **Simulating Attacks**: When testing a watermark, the backend applies selected image distortions (like rotating the image by 15 degrees or compressing it as a JPEG) and returns the distorted image to the frontend.

How the Frontend Works

1. **Interactivity**: The user can type text prompts to generate new images using a public API.
2. **State Sharing**: The uploader state is shared across tabs. If a user applies a rotation attack to a watermarked image, they can click "Send to Detector for Testing." The app automatically copies the distorted image to the detector tab and switches active panels, allowing the user to check if the ViT detector still recognizes the watermark.
3. **Responsive Styles**: All layouts stack vertically on small mobile displays and expand side-by-side on desktop screens.

---

3. Ethical Considerations Implemented (So Far)

Artificial Intelligence tools can make mistakes. In content verification (proving if an image is real or fake), outputting a raw "Yes" or "No" can lead to false accusations of plagiarism or content forging. We implemented the following ethical safeguards:

Surfacing Uncertainty (The 75% Rule)

Instead of forcing the detector model to output an absolute binary verdict, the system checks the model's confidence probability:

- **High Confidence (\ge 75\%)**: The app outputs a clear message like `"Likely watermarked"` or `"Likely not watermarked"`.
- **Low Confidence (< 75\%)**: The app outputs an `"Uncertain"` warning. This alerts the user that the model cannot guarantee its decision, indicating they should not rely on it as definitive proof.

Visual Transparency

The interface displays a clear notification explaining that uncertainty exists, educating users on the limitations of AI classifications.

Imperceptibility Safeguards

During the watermarking process, we set the embedding scale to 0.3. This ensures that the embedded signal is completely invisible to human observers, preserving the artistic integrity and quality of the original creator's image.

---

4. Ethical Considerations for Advanced Generative Models

As generative models like **GANs (Generative Adversarial Networks)** and **Diffusion Models** (e.g. Stable Diffusion, Midjourney) become more advanced, the risk of deepfakes and visual disinformation grows. Future watermarking systems must adopt stronger techniques:

A. Latent-Space Watermarking

Instead of adding a watermark to the pixels *after* the image is generated, we can embed the watermark directly during the image generation process:

- **How it works**: For Diffusion models, we can inject the watermark signature into the noise schedule or the latent space representations while the model is denoising the image.
- **Why it is ethical**: The watermark becomes part of the image's core structure (its "DNA"). This makes it almost impossible for users to strip or erase the watermark by editing the pixels.

B. Robustness Against Removal (Adversarial Training)

Bad actors will try to remove watermarks by cropping, filtering, or compressing images:

- **The Solution**: During model training, we must expose the decoder to heavily distorted images. This "adversarial training" ensures the decoder can still recognize the watermark even if the image is edited or screenshotted.

C. Watermark Spoofing Prevention (Cryptography)

Attackers might try to fake watermarks on real human photos to make them appear AI-generated:

- **The Solution**: We can use **cryptographic signatures** (private/public key pairs) to sign watermarks. Only authorized watermarking tools can apply a valid cryptographic seal, ensuring attackers cannot spoof or forge watermark signatures.

---

5. Ethical Deployment Protocols

If this application is deployed for public use, we must establish deployment safety protocols:

1. **Standard Standardization (C2PA)**

Deployments should align with open industry standards like the Coalition for Content Provenance and Authenticity (C2PA). This ensures the AI metadata is readable by standard web browsers, search engines, and social media platforms.

2. **Privacy Protections**

The 32-bit signature key should contain metadata identifying the generator system, but **must never** contain sensitive user data (like user IDs, email addresses, or locations) that could compromise user privacy if decoded by others.

3. **API Rate Limiting**

API endpoints should be protected with rate limiting to prevent malicious bots from sending millions of queries to reverse-engineer and break the watermarking algorithm.

4. **Human-in-the-Loop Oversight**

Watermark detection should never serve as the sole decider in critical situations (such as court evidence or news reporting). It should serve as an assistive tool, with final verification requiring human expert review.